22-10-2026

CO2 -ASSSESMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

COURSE NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: R SAI HASNIKA **REG NO:192424178**

QUESTION 1: Morphological Analysis for a Document Indexing System

Description of the question

The question asks us to analyze the words connected, connecting, and connection for a document indexing system.

A document indexing system stores searchable representations of words so that users can retrieve documents even when their queries use different word forms.

For example, the words connected and connecting come from the verb connect. By identifying their common base, a search engine can associate them during retrieval.

The program must:

- Separate the root word from the suffix.
- Identify whether the suffix is inflectional or derivational.
- Normalize related words into a common base form.
- Display the original word, root, suffix, classification, and normalized form.

Here, -ed and -ing are treated as inflectional suffixes, while -ion in connection is treated as derivational. The spelling change from connect to connection is represented using an explicit rule.

Algorithm

1. Start the program.
2. Store the input words in a list.
3. Define rules for recognizing the suffixes -ed, -ing, and -ion.
4. Check each word against the suffix rules.
5. Remove the suffix and recover the root using the corresponding rule.
6. Classify -ed and -ing as inflectional and -ion as derivational.

7. Set the normalized form to connect.
8. Display the analysis in a formatted table.
9. Stop the program.

Pseudocode

```
START
  INPUT words = connected, connecting, connection
  FOR each word:
    IF word ends with "ed":
      root = connect
      suffix = ed
      type = Inflectional
    ELSE IF word ends with "ing":
      root = connect
      suffix = ing
      type = Inflectional
    ELSE IF word is "connection":
      root = connect
      suffix = ion
      type = Derivational
    normalized_form = connect
    DISPLAY word, root, suffix, type, normalized_form
  END FOR
STOP
```

Code:

```python
words = ["connected", "connecting", "connection"]
analysis = {
    "connected":  ("connect", "ed",  "Inflectional", "connect"),
    "connecting": ("connect", "ing", "Inflectional", "connect"),
    "connection": ("connect", "ion", "Derivational", "connect")
}
print("\nMORPHOLOGICAL ANALYSIS - DOCUMENT INDEXING")
print("=" * 86)
print(f"{'Word':<16}{'Root':<16}{'Suffix':<12}"
      f"{'Type':<18}{'Normalized Form':<20}")
print("-" * 86)
for word in words:
    root, suffix, kind, normalized = analysis[word]
    print(f"{word:<16}{root:<16}{suffix:<12}"
          f"{kind:<18}{normalized:<20}")
print("=" * 86)
```

Output

```
MORPHOLOGICAL ANALYSIS - DOCUMENT INDEXING
======================================================================================
Word            Root            Suffix      Type              Normalized Form
--------------------------------------------------------------------------------------
connected       connect         ed          Inflectional      connect
connecting      connect         ing         Inflectional      connect
connection      connect         ion         Derivational      connect
======================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

Result

The program successfully identifies the roots and suffixes of the given words, classifies their morphological transformations, and normalizes all three words to connect for document indexing and retrieval.

QUESTION 2: Morphological Parsing for Sentiment Analysis

Description of the question

The question asks us to analyze unhappy, happiness, and happily.

In sentiment analysis, computers analyze text to identify opinions, feelings, and emotions. Words derived from the same base may express related meanings, so morphological parsing helps group them.

- unhappy contains the prefix un- and base happy.
- happiness contains the base happy and suffix -ness.
- happily contains the base happy and suffix -ly.

The prefix un- changes the meaning to its opposite. The suffixes -ness and -ly create related word forms. All three transformations are derivational rather than inflectional.

For this program, the normalized root is happy.

Algorithm

1. Start the program.
2. Store the words unhappy, happiness, and happily.
3. Define their prefixes, suffixes, and base forms.
4. Identify the prefix or suffix present in each word.
5. Separate the base form from the affix.
6. Classify the transformation as derivational.
7. Normalize each word to happy.
8. Display the results in a formatted table.
9. Stop the program.

Pseudocode

```
START
  INPUT words = unhappy, happiness, happily
  FOR each word:
    IF word starts with "un":
      prefix = un
      suffix = None
      base = happy
    ELSE IF word ends with "ness":
      prefix = None
```

suffix = ness

base = happy

ELSE IF word ends with "ly":

prefix = None

suffix = ly

base = happy

type = Derivational

normalized_form = happy

DISPLAY word, prefix, base, suffix, type

END FOR

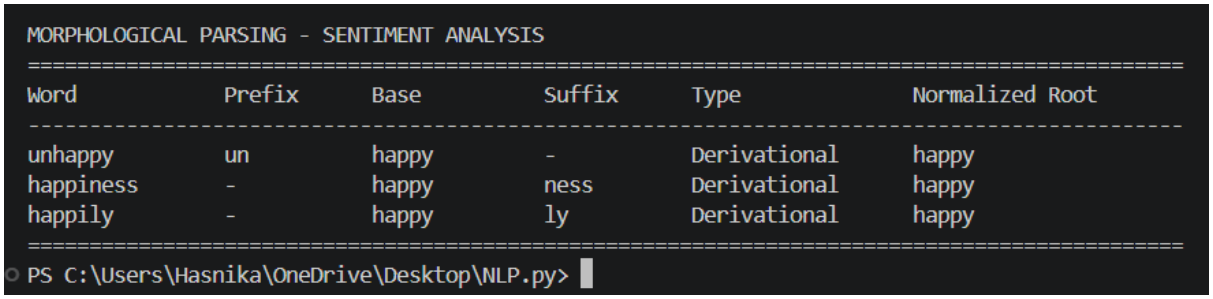
STOP

Code

co2-at12.py > ...

```
words = ["unhappy", "happiness", "happily"]
analysis = {
    "unhappy":   ("un", "-", "happy", "Derivational", "happy"),
    "happiness": ("-", "ness", "happy", "Derivational", "happy"),
    "happily":   ("-", "ly", "happy", "Derivational", "happy")
}
print("\nMORPHOLOGICAL PARSING - SENTIMENT ANALYSIS")
print("=" * 94)
print(f"{'Word':<16}{'Prefix':<12}{'Base':<14}"
      f"{'Suffix':<12}{'Type':<18}{'Normalized Root':<20}")
print("-" * 94)
for word in words:
    prefix, suffix, base, kind, normalized = analysis[word]
    print(f"{word:<16}{prefix:<12}{base:<14}"
          f"{suffix:<12}{kind:<18}{normalized:<20}")
print("=" * 94)
```

Output



Result

The program successfully identifies the prefixes, suffixes, and base forms of the given words. It classifies the transformations as derivational and normalizes the words to happy, supporting the grouping of related word forms during sentiment analysis. Note that normalization alone does not determine whether the sentiment is positive or negative.

QUESTION 3: Stemming-Based Preprocessing for a Search Engine

Description of the question

The question asks us to process played, player, and playing using a stemming-based preprocessing module.

A search engine receives queries and searches for matching documents. Different forms of a word can make searching difficult. Stemming reduces words to a shared stem so that related terms can be indexed more consistently.

The given words are formed as follows:

- played = play + -ed
- player = play + -er
- playing = play + -ing

The suffixes -ed and -ing are inflectional in these examples because they indicate grammatical forms of the verb. The suffix -er is derivational because it forms the noun player, referring to a person who plays.

The intended common indexing form is play. A rule-based program is used here so that all three examples have the required common stem.

Algorithm

1. Start the program.
2. Store the input words.
3. Define the suffix rules for -ed, -er, and -ing.
4. Check each word for a matching suffix.
5. Remove the suffix and recover the stem play.
6. Classify -ed and -ing as inflectional and -er as derivational.
7. Use play as the normalized indexing form.
8. Display the original word, stem, removed affix, transformation type, and normalized form.
9. Stop the program.

Pseudocode

```
START
  INPUT words = played, player, playing
  FOR each word:
    IF word ends with "ed":
      stem = play
      affix = ed
      type = Inflectional
    ELSE IF word ends with "er":
      stem = play
      affix = er
      type = Derivational
    ELSE IF word ends with "ing":
      stem = play
      affix = ing
      type = Inflectional
```

normalized_form = play

DISPLAY word, stem, affix, type, normalized_form

END FOR

STOP

Code

```python
words = ["played", "player", "playing"]
analysis = {
    "played":  ("play", "ed",  "Inflectional", "play"),
    "player":  ("play", "er",  "Derivational", "play"),
    "playing": ("play", "ing", "Inflectional", "play")
}
print("\nSTEMMING-BASED SEARCH ENGINE PREPROCESSING")
print("=" * 91)
print(f"{'Word':<16}{'Stem':<14}{'Removed Affix':<17}"
      f"{'Type':<18}{'Normalized Form':<18}")
print("-" * 91)
for word in words:
    stem, affix, kind, normalized = analysis[word]
    print(f"{word:<16}{stem:<14}{affix:<17}"
          f"{kind:<18}{normalized:<18}")
print("=" * 91)
```

Output

```
STEMMING-BASED SEARCH ENGINE PREPROCESSING
===========================================================================================
Word            Stem          Removed Affix    Type              Normalized Form
-------------------------------------------------------------------------------------------
played          play          ed               Inflectional      play
player          play          er               Derivational      play
playing         play          ing              Inflectional      play
===========================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

Result

The program successfully extracts the intended stem play from all three words. It distinguishes grammatical inflections from derivational word formation and produces a common normalized form for search engine indexing.

QUESTION 4: Finite-State Morphological Parsing for Verb Inflections

Description of the question

The question asks us to design a finite-state morphological parser for writes, writing, and written.

A finite-state machine (FSM) consists of states and transitions. It can process a word character by character and move between states according to predefined rules.

In morphological parsing, states can represent stages such as:

- Start: begin processing the word.
- Root recognition: identify the verb root write.
- Suffix recognition: identify a suffix or irregular form.
- Accept: finish parsing successfully.

The words are analyzed as follows:

- writes = write + -s, a regular third-person singular inflection.
- writing = write + -ing, a regular present participle with the final e removed before adding the suffix.
- written is an irregular past participle of write.

All three are normalized to the root write. The program uses explicit transitions and an irregular-form mapping for written.

Algorithm

1. Start the program.
2. Store writes, writing, and written.
3. Define the FSM states: START, ROOT, SUFFIX, and ACCEPT.
4. Define the regular suffix rules for -s and -ing.
5. Define the irregular mapping written → write.
6. Process each input word using the applicable transition path.
7. Record the sequence of states visited.
8. Identify the root, morphological components, and normalized form.
9. Classify each word as a regular or irregular inflection.
10. Display the results in a formatted table.

11. Stop the program.

Pseudocode

```
START
   INPUT words = writes, writing, written
   DEFINE states START, ROOT, SUFFIX, ACCEPT
   FOR each word:
      state = START
      IF word is "writes":
         START -> ROOT -> SUFFIX -> ACCEPT
         root = write
         suffix = s
         type = Regular
      ELSE IF word is "writing":
         START -> ROOT -> SUFFIX -> ACCEPT
         root = write
         suffix = ing
         type = Regular
      ELSE IF word is "written":
         START -> ROOT -> SUFFIX -> ACCEPT
         root = write
         suffix = irregular en
         type = Irregular
      normalized_form = write
      DISPLAY state path, components, root, type
   END FOR
STOP
```

Code

```python
words = ["writes", "writing", "written"]
analysis = {
    "writes": (
        "write", "write + s", "s",
        "Regular", "START -> ROOT -> SUFFIX -> ACCEPT"
    ),
    "writing": (
        "write", "write + ing", "ing",
        "Regular", "START -> ROOT -> SUFFIX -> ACCEPT"
    ),
    "written": (
        "write", "write + irregular participle", "irregular",
        "Irregular", "START -> ROOT -> IRREGULAR -> ACCEPT"
    )
}
print("\nFINITE-STATE MORPHOLOGICAL PARSING")
print("=" * 119)
print(f"{'Word':<12}{'Components':<31}{'Root':<10}"
      f"{'Class':<13}{'Normalized':<13}{'State Transition Path'}")
print("-" * 119)
for word in words:
    root, components, suffix, kind, path = analysis[word]
    print(f"{word:<12}{components:<31}{root:<10}"
          f"{kind:<13}{root:<13}{path}")
print("=" * 119)
```

Output

```
FINITE-STATE MORPHOLOGICAL PARSING
=======================================================================================================================
Word        Components                      Root      Class        Normalized    State Transition Path
-----------------------------------------------------------------------------------------------------------------------
writes      write + s                       write     Regular      write         START -> ROOT -> SUFFIX -> ACCEPT
writing     write + ing                     write     Regular      write         START -> ROOT -> SUFFIX -> ACCEPT
written     write + irregular participle    write     Irregular    write         START -> ROOT -> IRREGULAR -> ACCEPT
=======================================================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

Result

The program successfully represents the morphological parsing of the three verb forms using finite-state transition paths. It identifies the root write, distinguishes regular forms from the irregular form written, and normalizes all words to write.

QUESTION 5: Porter Stemmer for Document Retrieval

Description of the question

The question asks us to implement a Porter Stemmer-based preprocessing module for the words relational, relation, and relate.

The Porter Stemming algorithm is a rule-based algorithm that reduces words by removing or transforming suffixes. It is widely used in information retrieval and text preprocessing.

The given words contain related forms of the concept relate:

- relational contains a derivational ending.
- relation contains a noun-forming ending.
- relate is the original verb form.

The Porter algorithm applies suffix transformations in stages. For these examples, the final Porter stem is relat for all three words. A stem does not need to be a complete dictionary word.

The program will display the original word, the main stemming rule, an intermediate form, and the final stem.

Algorithm

1. Start the program.
2. Store the input words.
3. Initialize a Porter Stemmer.
4. Process each word according to the Porter stemming rules.
5. For relational, change -ational to -ate, giving relate.
6. For relation, remove the eligible -ion ending, giving relat.
7. For relate, remove the final e, giving relat.
8. Apply any remaining Porter rules needed to obtain the final stem.
9. Display the original word, rule, intermediate form, and final stem.
10. Stop the program.

Pseudocode

START

INPUT words = relational, relation, relate

INITIALIZE Porter Stemmer

FOR each word:

IF word is "relational":

intermediate = relate

rule = ational -> ate

ELSE IF word is "relation":

intermediate = relat

rule = remove ion

ELSE IF word is "relate":

intermediate = relat

rule = remove final e

final_stem = PorterStemmer(word)

DISPLAY word, rule, intermediate, final_stem

END FOR

STOP

Code

co2-at15.py > ...

```python
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
words = ["relational", "relation", "relate"]
steps = {
    "relational": ("ational -> ate", "relate"),
    "relation":   ("remove ion", "relat"),
    "relate":     ("remove final e", "relat")
}
print("\nPORTER STEMMER - DOCUMENT RETRIEVAL")
print("=" * 83)
print(f"{'Original Word':<18}{'Stemming Rule':<22}"
      f"{'Intermediate Form':<22}{'Final Stem':<15}")
print("-" * 83)
for word in words:
    rule, intermediate = steps[word]
    final_stem = stemmer.stem(word)
    print(f"{word:<18}{rule:<22}"
          f"{intermediate:<22}{final_stem:<15}")
print("=" * 83)

```

Output

```
PORTER STEMMER - DOCUMENT RETRIEVAL
================================================================================
Original Word     Stemming Rule         Intermediate Form     Final Stem
--------------------------------------------------------------------------------
relational        ational -> ate        relate                relat
relation          remove ion            relat                 relat
relate            remove final e        relat                 relat
================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

Result

The program successfully applies Porter stemming to the given words and produces the common stem relat. It demonstrates how stemming reduces related word forms to a shared representation for document indexing and information retrieval.